

Introduction

The Question

Graph Convolutional Networks stack multiple layers of propagation, weight matrices, and ReLU activations. Are all of those nonlinearities actually pulling their weight?

The SGC Hypothesis

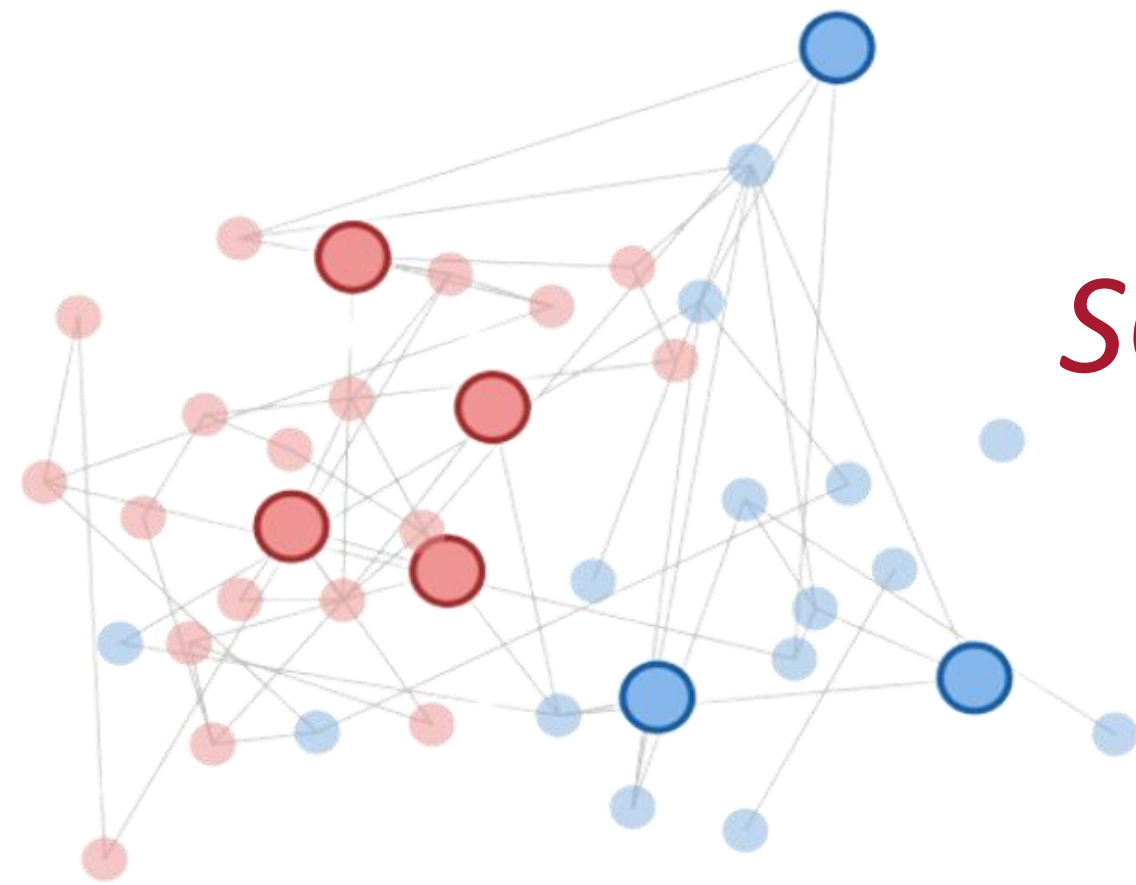
Wu et al. (2019) argue: remove the ReLUs, and the entire K-layer GCN collapses into a single linear operator that can be precomputed once.

Our Goal

Achieve improved training speed and comparable accuracy between SGC and GCNs to challenge the need for nonlinearities.

A training-scale graph: 40 nodes, 7 labels, two communities

cross-class edges create the hard cases the GCN must resolve



● UNDERGRAD (true class) ● GRAD (true class)
● UNDERGRAD (labeled) ● GRAD (labeled)

Fig 1: Example graph. Large nodes are labeled, small nodes are unlabeled.

Materials

Datasets (Sen et al., 2008)

Cora — 2,708 nodes · 5,429 edges
Citeseer — 3,327 nodes · 4,732 edges
Pubmed — 19,717 nodes · 44,338 edges

Splits: Public (Sen et al.) — 20 train/class, 500 val, 1000 test.

PyTorch + PyTorch Geometric
Adam (lr=0.2 SGC, lr=0.01 GCN)
100 epochs SGC, 200 with early stopping GCN
10 random seeds per (model, dataset)

Hardware

Google Colab T4

Replicating Simple Graph Convolution

A Linear, Precomputable Alternative to Graph Convolutional Networks

Neha Arora · Sol Jung · Shashank Kalyanaraman · Kabir Samsi

Cornell University · CS 4782: Introduction to Deep Learning

Based on Wu et al. (2019), ICML

Methodology

GCN: $\text{softmax}(S \sigma(S \sigma(SX\Theta_1) \Theta_2) \Theta_3)$ SGC: $\text{softmax}(S^K X \Theta)$
 $S = \tilde{D}^{-1/2}(A + I)\tilde{D}^{-1/2}$ is the augmented normalized adjacency. We use $K = 2$.

GCN (k layers, non-linear)

Each Epoch: K-sparse matrix multiplication + K nonlinearities



SGC

(linear, K-step prop. precomputed)

Remove σ, Collapse Θ, Precompute S^K

Each Epoch: 1 dense matrix multiplication (cheap!)

PRECOMPUTE: no parameters, done once

TRAIN: logistic regression on X

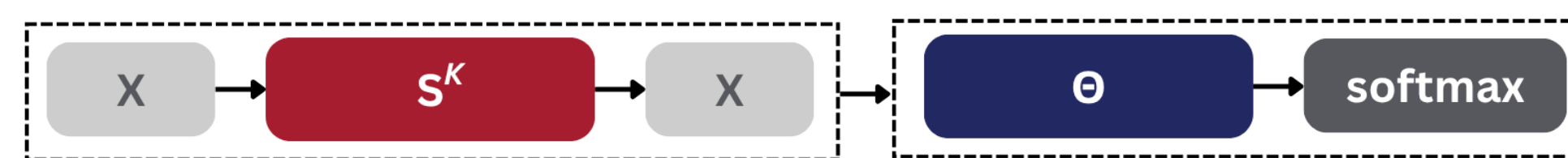


Fig 2: SGC simplifies GCN by removing nonlinearities and precomputing graph propagation once.

SGC is way cheaper ... and we have the receipts

ORDER #GCN	ORDER #SGC
200 EPOCHS (EARLY STOP)	100 EPOCHS(150 CITESEER)
LR = 0.01	LR = 0.2
---	---
PER EPOCH	ONE-TIME ONLY
SPARSE MAT-MUL × K..... K OPS	S ² X PRECOMPUTE 1 OP
RELU ACTIVATION × K..... K OPS	PER EPOCH
WEIGHT MATRIX × K..... K OPS	DENSE MAT-MUL × 1..... 1 OP
	RELU ACTIVATION
	BACKPROP THRU GRAPH
---	---
OUR RESULTS	OUR RESULTS
TRAIN TIME (CORA) 0.763S	TRAIN TIME (CORA) 0.274S
TRAIN TIME (CITESEER) .. 0.794S	TRAIN TIME (CITESEER)... 0.516S
TRAIN TIME (PUBMED) 0.693S	TRAIN TIME (PUBMED).... 0.321S
TEST ACC (CORA) 81.6 ± 0.6%	TEST ACC (CORA) 79.9 ± 0.2%
TEST ACC (CITESEER) ... 70.8 ± 0.8%	TEST ACC (CITESEER)... 71.9 ± 0.2%
TEST ACC (PUBMED) ... 78.9 ± 0.7%	TEST ACC (PUBMED)..... 78.7 ± 0.1%
	SPEEDUP VS GCN ~2.2×

Fig 3: Final results and per-epoch cost comparison between GCN and SGC.

Results

Replication Outcome

SGC matches GCN within ~1% on Cora and Pubmed, and edges out GCN on Citeseer (71.9% vs 70.8%) — consistent with the paper's reduced-overfitting argument.

Timing

GC trains 1.5–2.8 × faster on CPU. The paper reports 28 × on GPU (GTX 1080 Ti) — the gap is expected since SGC's advantage is in per-epoch cost, which scales with hardware

Why It Works

In homophilic graphs, important information comes from nearby similar nodes
--> SGC acts like a low-pass filter

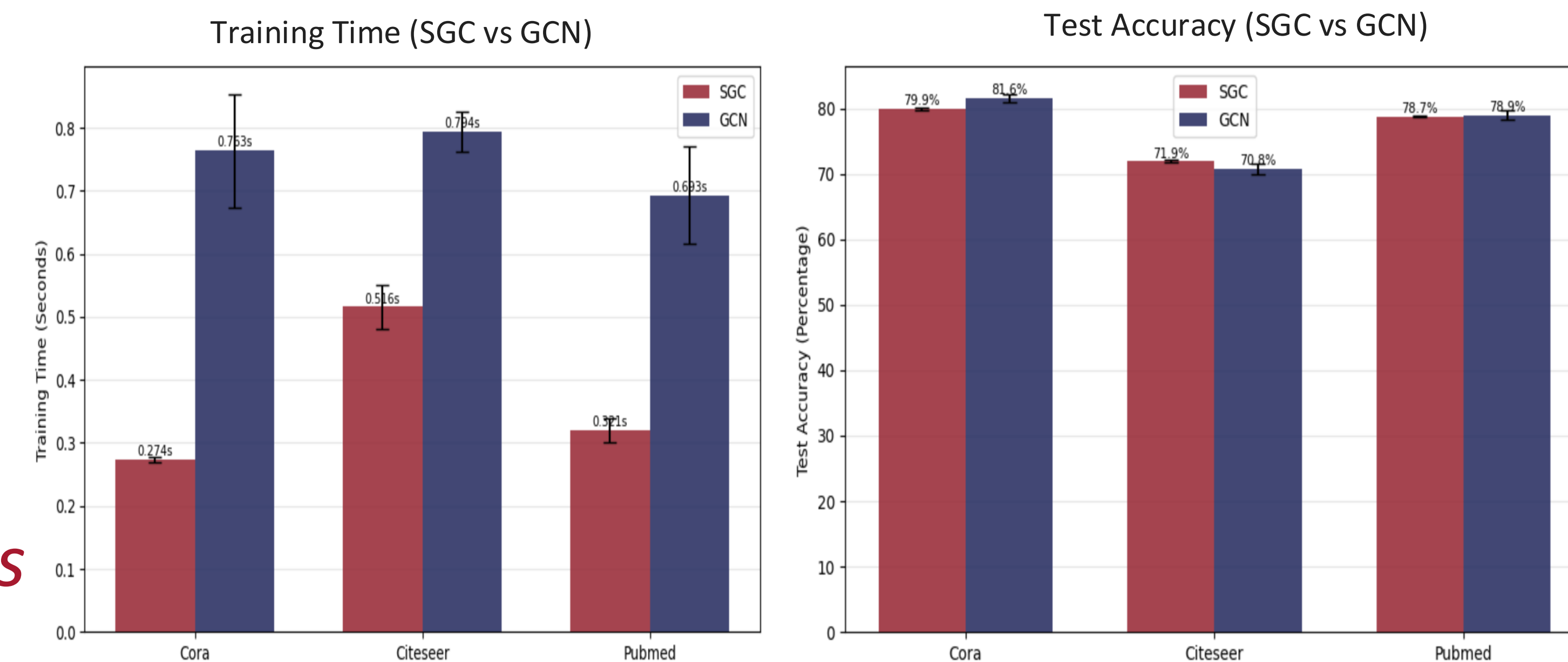


Fig 4: Bar charts comparing Training Time and Test Accuracy between GCN and SGC on Cora, Pubmed, and Citeseer.

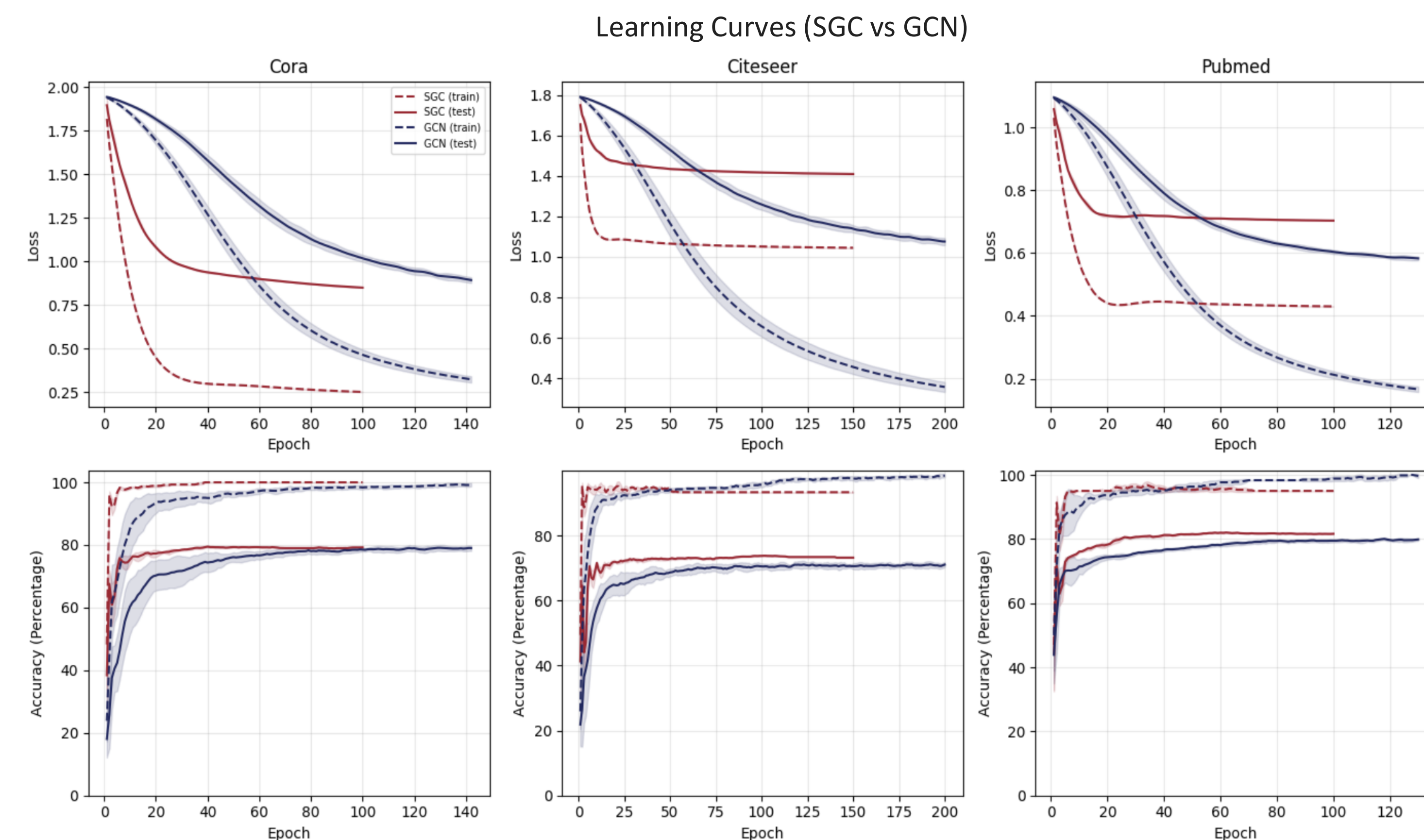


Fig 5: Performance over epochs. SGC converges faster.